МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ» (ФГБОУ ВО
«ВГУ»)

Факультет прикладной математики, информатики и механики
Кафедра математического обеспечения ЭВМ

**Цифровая обработка изображений**

Лабораторная работа № 2

Выделение контуров и подавление шума
Варианты 1, 2 и 3

Студент: _____
Группа: _____
Преподаватель: Сырых А. С.

Воронеж, 2026

# Содержание

1. Постановка задачи..... 3
2. Математическое описание алгоритма.....4
2.1. Алгоритмы выделения контуров.....4
2.2. Линейные усредняющие фильтры..... 6
2.3. Порядковые фильтры..... 8
3. Программное описание алгоритма.....10
4. Результат работы программы.....13
5. Заключение.....22
Приложение А.....23
Приложение Б.....25
Приложение В..... 27

## 1. Постановка задачи

Реализовать на языке C++17 обработку полутоновых изображений по всем трём вариантам задания.

Выполнить выделение контуров на изображении с применением следующих операторов:

1. Оператор Лапласа;
2. Оператор Робертса;
3. Оператор Собела.

Реализовать подавление гауссова шума на изображениях в пространственной области усредняющими фильтрами: арифметическим с окнами 3×3 и 5×5 и гауссовым с окном 5×5 при $\sigma=1$.

Реализовать обработку изображений с гауссовым шумом порядковыми фильтрами: медианным, минимума и максимума с окнами 3×3 и 5×5.

Завершить заданный класс Image: обеспечить чтение и сохранение PNG, доступ к пикселям, поддержку одного–четырёх каналов и независимое копирование изображений. Сохранить интерфейс класса без добавления новых методов.

Подготовить тестовые изображения, демонстрирующие работу и различия методов. Проверить корректность программы тестами, сравнить результаты с чистыми эталонами по MSE и PSNR и сохранить результаты обработки в отдельные файлы.

## 2. Математическое описание алгоритма

Изображение рассматривается как двумерная дискретная функция f(x,y), где W и H обозначают ширину и высоту растра, а f — яркость пикселя в пределах 0–255. Цветной вход предварительно переводится в полутоновое представление по формуле (1). Для одно- и двухканального входа используется существующий канал яркости.

$$f(x, y) = \mathrm{round}(0.299R + 0.587G + 0.114B) \qquad (1)$$

При обращении за пределы растра берётся ближайший допустимый пиксель. Такое продолжение сохраняет постоянный фон у границ. Каждый метод читает исходный массив и формирует отдельное выходное изображение.

### 2.1. Алгоритмы выделения контуров

Контуры соответствуют резким локальным изменениям яркости, которые выявляются с помощью производных первого и второго порядков. В дискретном изображении они заменяются разностными приближениями.

#### Оператор Собела

Оператор вычисляет градиент яркости, одновременно выполняя локальное сглаживание поперёк направления дифференцирования. Для этого применяются две пространственные маски размера 3×3 (2):

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$
$$S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \qquad (2)$$

Компоненты градиента и его амплитуда определяются по (3). Суммирование ведётся по смещениям i,j = −1,0,1 относительно текущего пикселя:

$$G_x = \sum_{i=-1}^{1}\sum_{j=-1}^{1} S_x[j,i] f(x+i, y+j)$$
$$G_y = \sum_{i=-1}^{1}\sum_{j=-1}^{1} S_y[j,i] f(x+i, y+j) \qquad (3)$$
$$M(x,y) = \sqrt{G_x^2 + G_y^2}$$

Для записи в восьмибитный канал результат ограничивается и округляется по (4). Эта операция не является растяжением яркостей по максимуму конкретного изображения. Она также не выполняет пороговую бинаризацию:

$$Q(t) = \mathrm{round}(\min(255, \max(0, t)))$$
$$g(x,y) = Q(M(x,y)) \qquad (4)$$

*Алгоритм 1. Оператор Собела*

1. Получить полутоновый вход по формуле (1).
2. Для каждого пикселя найти Gx и Gy с использованием масок (2).
3. Вычислить амплитуду по (3) и подготовить значение канала по (4).
4. Поместить значение в отдельное результирующее изображение.

**Оператор Робертса**

Используются диагональные конечные разности в компактном окне 2×2. Этому соответствуют две маски (5):

$$R_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad R_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \qquad (5)$$

Из этих масок следуют две разности (6). Амплитуда получается как корень из суммы их квадратов, после чего применяется ограничение (4):

$$a = f(x,y) - f(x+1, y+1)$$
$$b = f(x+1, y) - f(x, y+1) \qquad (6)$$
$$M(x,y) = \sqrt{a^2 + b^2}$$

*Алгоритм 2. Оператор Робертса*

1. Сформировать яркостное изображение по (1).

2. В каждом окне 2×2 вычислить диагональные разности по (6).

3. Найти амплитуду отклика и записать её с учётом (4).

**Оператор Лапласа**

Лапласиан является оператором второй пространственной производной. В непрерывной форме он определяется суммой вторых производных по координатам:

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \qquad (7)$$

В работе применяется четырёхсвязное дискретное приближение с ядром (8):

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \qquad (8)$$

Поскольку вторая разность может иметь любой знак, для отображения используется её модуль. Формула расчёта приведена в (9):

$$\begin{gathered} L = f(x-1,y) + f(x+1,y) \\ + f(x,y-1) + f(x,y+1) - 4f(x,y) \qquad (9) \\ g(x,y) = Q(|L|) \end{gathered}$$

*Алгоритм 3. Оператор Лапласа*

1. Преобразовать вход в полутоновое представление по (1).

2. Вычислить вторую разность с использованием маски (8).

3. Взять модуль отклика и сформировать результат по (9).

### 2.2. Линейные усредняющие фильтры

Усредняющие фильтры предназначены для ослабления аддитивного гауссова шума за счёт объединения значений в окрестности пикселя. Сглаживание затрагивает и полезные перепады яркости, поэтому размер окна влияет на сохранение деталей.

**Прямоугольный усредняющий фильтр (Box Blur)**

Яркость пикселя заменяется средним арифметическим значений его окрестности размером k×k. Радиус окна r=(k−1)/2, а число элементов равно k². Вычисление выполняется по формуле (10):

$$g(x,y) = Q\left(\frac{1}{k^2}\sum_{i=-r}^{r}\sum_{j=-r}^{r} f(x+i, y+j)\right) \qquad (10)$$

*Алгоритм 4. Прямоугольное усреднение*

1. Выбрать нечётный размер окна и определить его радиус.
2. Присвоить каждому из k² элементов вес 1/k².
3. Вычислить сумму значений окрестности с этими весами по (10).
4. Округлить результат и сохранить его в новом изображении.

**Фильтр Гаусса (Gaussian Blur)**

Весовые коэффициенты задаются двумерной гауссовой функцией (11). Параметр σ определяет масштаб сглаживания: близкие к центру элементы получают больший вес, чем удалённые.

$$G(i,j) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{i^2+j^2}{2\sigma^2}\right) \qquad (11)$$

На конечном окне веса необходимо нормировать по (12). Общий множитель 1/(2πσ²) при делении сокращается, поэтому реализация вычисляет только экспоненциальную часть:

$$w(i,j) = \frac{G(i,j)}{\sum_{i=-r}^{r}\sum_{j=-r}^{r} G(i,j)} \qquad \sum_{i=-r}^{r}\sum_{j=-r}^{r} w(i,j) = 1 \qquad (12)$$

После подготовки ядра выход определяется взвешенной суммой (13):

$$g(x,y) = Q\left(\sum_{i=-r}^{r}\sum_{j=-r}^{r} w(i,j) f(x+i, y+j)\right) \qquad (13)$$

*Алгоритм 5. Гауссово усреднение*

1. Задать размер окна k и положительное значение σ.

2. Определить веса по (11) и нормировать их согласно (12).

3. Для каждого пикселя вычислить взвешенную сумму по (13).

4. Сформировать отдельный выходной массив с учётом (4).

### 2.3. Порядковые фильтры

Для каждого пикселя собираются $k^2$ яркостей нечётного окна. Порядковый фильтр выбирает одно из этих значений, а не вычисляет их среднее. Используются окна 3×3 и 5×5.

#### Медианный фильтр

Медиана — средний элемент упорядоченной выборки. Для девяти значений берётся пятое, для двадцати пяти — тринадцатое. Обозначим отсортированные значения $v_0,\ldots,v_{k^2-1}$:

$$g(x,y) = v_{\frac{k^2-1}{2}} \qquad (14)$$

#### Фильтр минимума

Выбирается наименьшая яркость окна. Тёмные области расширяются, светлые сужаются:

$$g(x,y) = \min_{\substack{-r\le i\le r\\ -r\le j\le r}} f(x+i, y+j) \qquad (15)$$

#### Фильтр максимума

Выбирается наибольшая яркость окна. Светлые области расширяются, тёмные сужаются:

$$g(x,y) = \max_{\substack{-r\le i\le r\\ -r\le j\le r}} f(x+i, y+j) \qquad (16)$$

*Алгоритм 6. Порядковая фильтрация*

1. Проверить вход и нечётный размер окна от 1 до 31.

2. Для каждого пикселя собрать значения окна, повторяя край при выходе за изображение.

3. Выбрать медиану, минимум или максимум по формулам (14)–(16).

4. Сохранить значение в отдельное изображение, не изменяя вход.

В коде медиана находится через std::nth_element: полная сортировка не требуется. Минимум и максимум вычисляются через std::min_element и std::max_element. Эти два фильтра меняют яркость и геометрию деталей, поэтому не гарантируют улучшения изображения с гауссовым шумом.

## 3. Программное описание алгоритма

Программная реализация выполнена на C++17. Поскольку по условию нельзя расширять класс Image, все алгоритмы обработки вынесены в пространство имён Filters. Класс Image сохраняет заданный интерфейс и отвечает за память, PNG и доступ к пикселям. Данные хранятся в std::unique_ptr<uint8_t[]>, а копирование создаёт независимый массив. Для серого входа getColor возвращает яркость в r, устанавливая g и b в ноль; setColor сохраняет альфа-канал. При недопустимых координатах доступ безопасно игнорируется.

Функция Filters::convertToGrayscale формирует одно яркостное изображение по (1). Полученный объект используется при обработке, а исходные данные не изменяются. Вспомогательная readBorderPixel повторяет ближайший граничный пиксель, writePixel реализует ограничение и округление по (4).

Filters::sobelEdges принимает изображение по константной ссылке, получает его полутоновое представление и вызывает общий внутренний расчёт calculateEdges с выбором Собела. В нём выполняется алгоритм 1: накапливаются компоненты по маскам (2), затем вычисляется амплитуда (3). Соответствующие фрагменты кода приведены в приложении А.

Filters::robertsEdges использует тот же внутренний расчёт с другим режимом. Для каждой позиции берутся две диагональные разности по (6); их амплитуда преобразуется в допустимый диапазон. Это реализация алгоритма 2, представленная в приложении А.

Filters::laplaceEdges выбирает четырёхсвязную вторую разность и её модуль согласно (8), (9). Функция соответствует алгоритму 3; её реализация и общий расчёт calculateEdges также приведены в приложении А.

Filters::meanFilter получает изображение и размер окна. Через внутреннюю функцию weightedAverage она задаёт равные веса и выполняет алгоритм 4 по (10). Filters::gaussianFilter дополнительно проверяет положительность и конечность σ, формирует нормированные гауссовы веса и

выполняет алгоритм 5 по (11)–(13). Эти фрагменты приведены в приложении Б.

Точка входа находится в Source.cpp. main выбирает вход и каталог результата, а runFilters последовательно запускает три контурные, две усредняющие и три порядковые функции обработки. Арифметическое среднее применяется с окнами 3×3 и 5×5, Гаусс — с окном 5×5 при σ=1. Порядковые фильтры применяются с окнами 3×3 и 5×5. Вместе с полутоновым входом записываются 13 выходных PNG. Сохранение проверяет saveImage; стандартные исключения перехватываются в main, сопровождаются сообщением и кодом завершения 1.

Без аргументов используется готовый файл test_input/shapes_noisy.png. Можно передать пути входа, каталога результата и необязательного чистого эталона либо --random с необязательным seed для дополнительной случайной сцены. Она состоит из градиента, прямоугольников и тонких штрихов, имеет размер 320×224 и шум со стандартным отклонением 18. Если готовый пример отсутствует, такой тест создаётся автоматически. Этот режим не используется для численных таблиц отчёта: основные шумные входы подготовлены в ImageMagick согласно условию варианта 2.

Функции medianFilter, minimumFilter и maximumFilter используют общий расчёт selectFromWindow и соответствуют алгоритму 6. Их реализация приведена в приложении В. Функции calculateMSE и calculatePSNR сравнивают результат с чистым эталоном. При наличии эталона runFilters сохраняет metrics.csv для шумного входа и девяти результатов подавления шума; контурные карты в эту оценку не включаются. Различие размеров эталона и входа вызывает ошибку до сохранения результатов.

В решении Visual Studio предусмотрены отдельные проекты Lab и Tests. Проверка содержит 21 тестовый случай и 384 условий, все пройдены. Проверяются загрузка и сохранение, каналы, прозрачность, копирование, граничный доступ и заранее рассчитанные ответы фильтров. Например, на

ступени 0→40 ответы Лапласа, Робертса и Собела равны соответственно 40, 57 и 160; среднее 3×3 для одиночного импульса 255 даёт 28.

## 4. Результат работы программы

Для эксперимента подготовлены два полутоновых изображения размером 256×192. В наборе А присутствуют крупные фигуры, ровный фон и плавное изменение яркости. В наборе Б размещены тонкие штрихи, клетки, окружность и диагональ. Первый набор показывает изменение шума на широких областях, второй позволяет обнаружить потерю небольших деталей. Для каждой сцены сохранены чистый эталон и шумная версия.

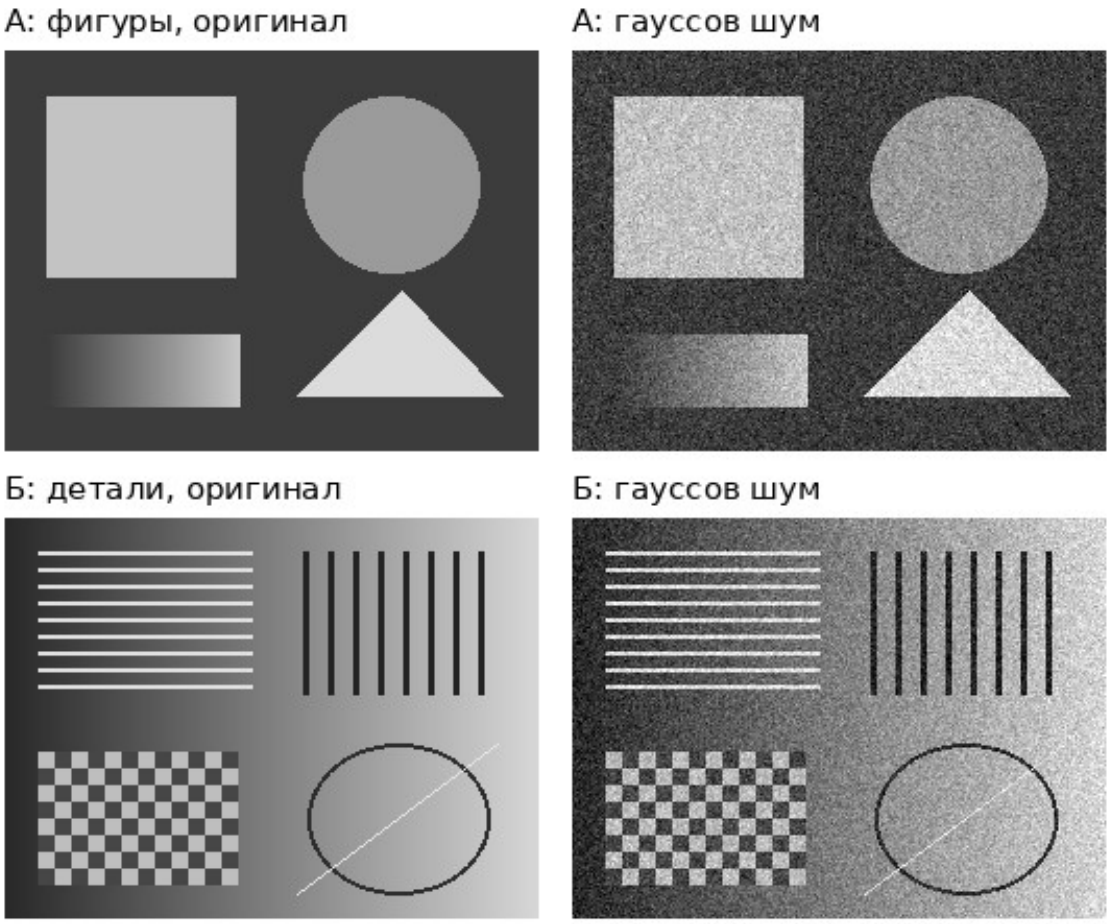



Рис. 1. Слева — чистые изображения А и Б, справа — соответствующие версии с гауссовым шумом.

Шум добавлен вне C++-программы редактором ImageMagick с параметрами +noise Gaussian и -attenuate 0.9, с последующим сохранением в восьмибитный PNG. Начальные значения генератора равны 731 и 1201. Возможность применять другой редактор прямо предусмотрена вариантом 2; для варианта 3 в исходном задании назван Photoshop. Приложенные тесты подготовлены в ImageMagick и не выдаются за обработанные в Photoshop.

Измеренное стандартное отклонение разности с чистым эталоном равно 17.92 для А и 17.92 для Б. Это характеристика шума в уровнях яркости; она не совпадает по смыслу с σ=1 пространственного гауссова фильтра.

Тестовые изображения подобраны так, чтобы содержать:

- мелкие детали и отчётливые границы — для проверки точности выделения контуров;
- однородные области — для оценки подавления случайных колебаний яркости;
- плавные переходы — для выявления искажений, возникающих при округлении.

Преимущества и ограничения методов оцениваются по результатам обработки и сопоставляются с математической моделью.

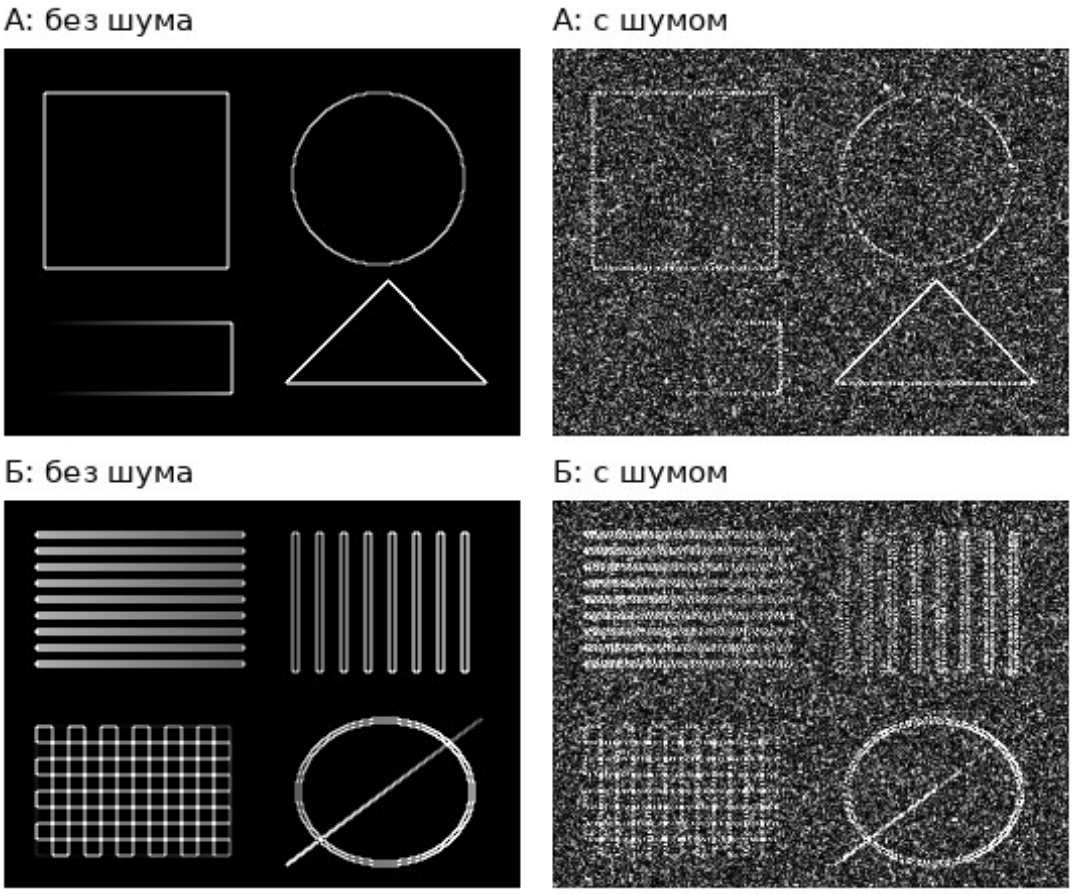



Рис. 2. Результат действия оператора Лапласа: сверху А, снизу Б; слева чистый вход, справа шумный.

Оператор Лапласа подчёркивает перепады яркости, но сильно реагирует и на шум. После взятия модуля становятся видимыми отклики

разных знаков по обе стороны границы, поэтому она может выглядеть двойной. На шумном фоне появляется множество дополнительных откликов.

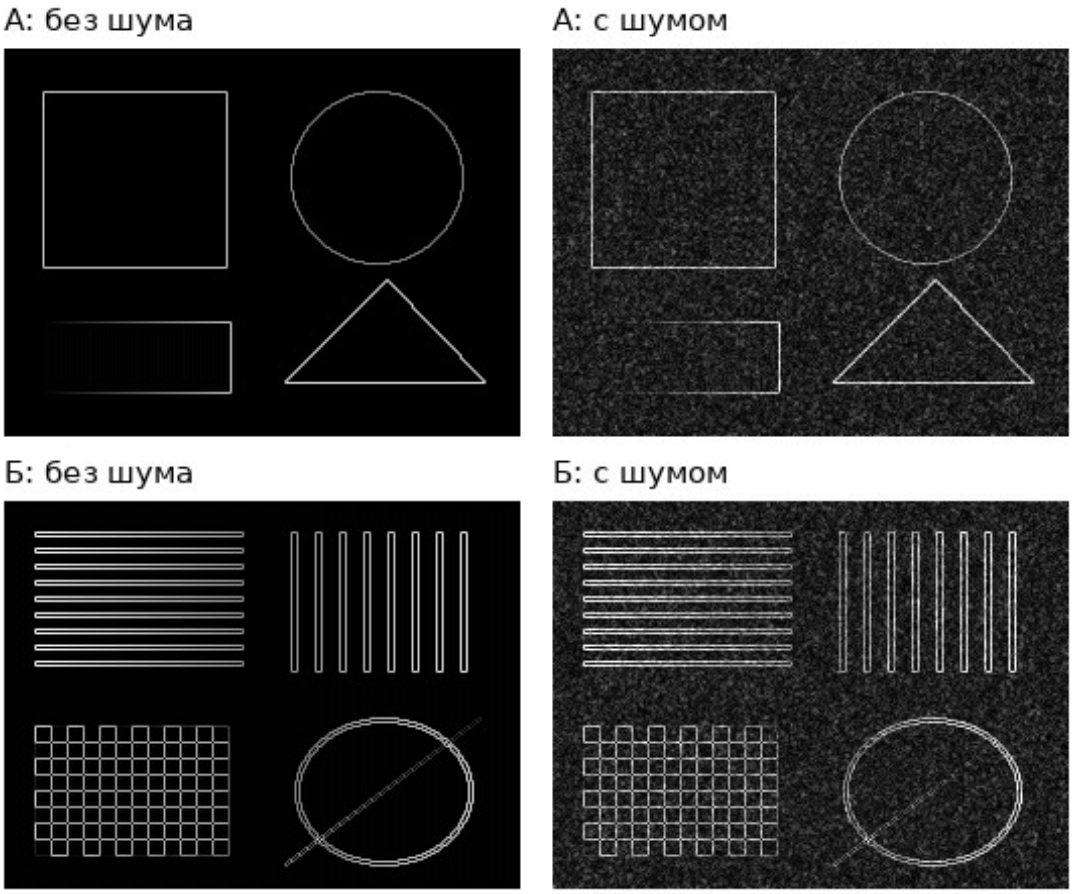



Рис. 3. Результат действия оператора Робертса в том же расположении.

Компактное окно 2×2 позволяет получить тонкие границы. Однако отсутствие предварительного сглаживания делает метод чувствительным к случайным перепадам: на однородных областях шумного входа возникают ложные контуры.

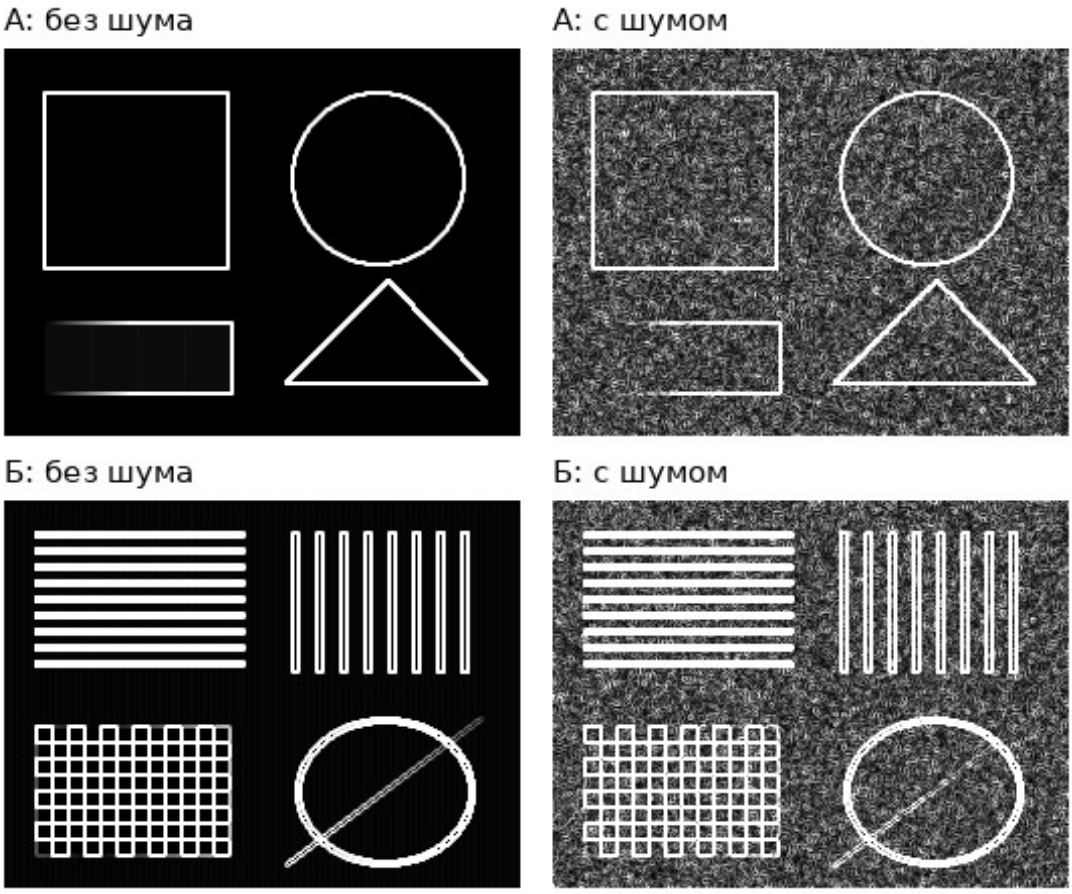



Рис. 4. Результат действия оператора Собела в том же расположении.

Собел совмещает оценку градиента со сглаживанием в поперечном направлении. Он формирует более широкий отклик, чем Робертс, но также выделяет шумовые изменения. Яркость разных контурных карт нельзя напрямую считать мерой качества: коэффициенты масок и степень насыщения у операторов различаются.

На наборе Б близкие границы тонких штрихов и клеток могут сливаться. Внутри линейного градиента вторая разность близка к нулю, а первые разности реагируют на плавное изменение яркости. Округление значений и ограничение сильных откликов до 255 влияют на отображаемый результат.

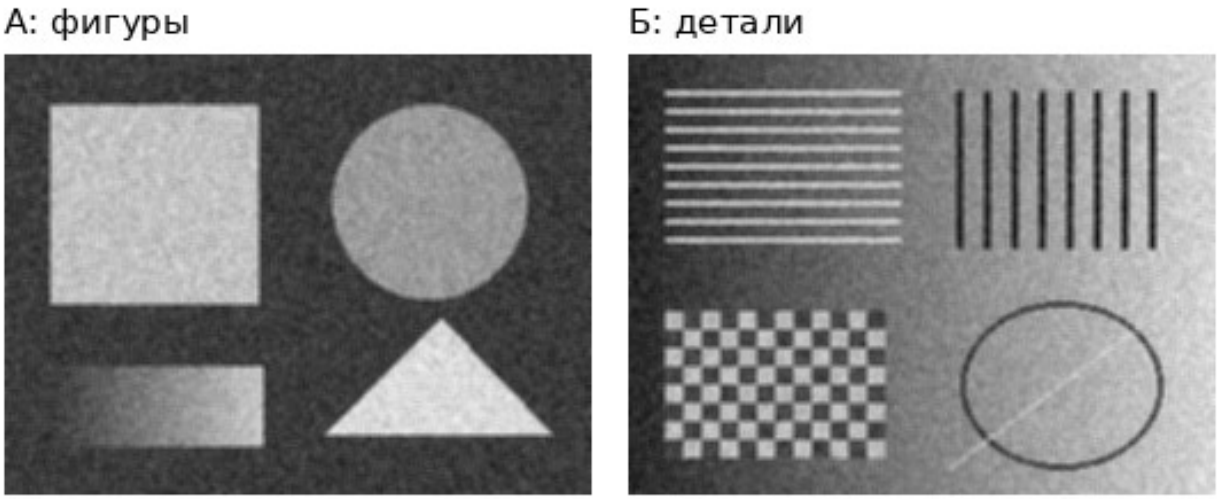



Рис. 5. Результат действия прямоугольного усредняющего фильтра 3×3: слева А, справа Б.

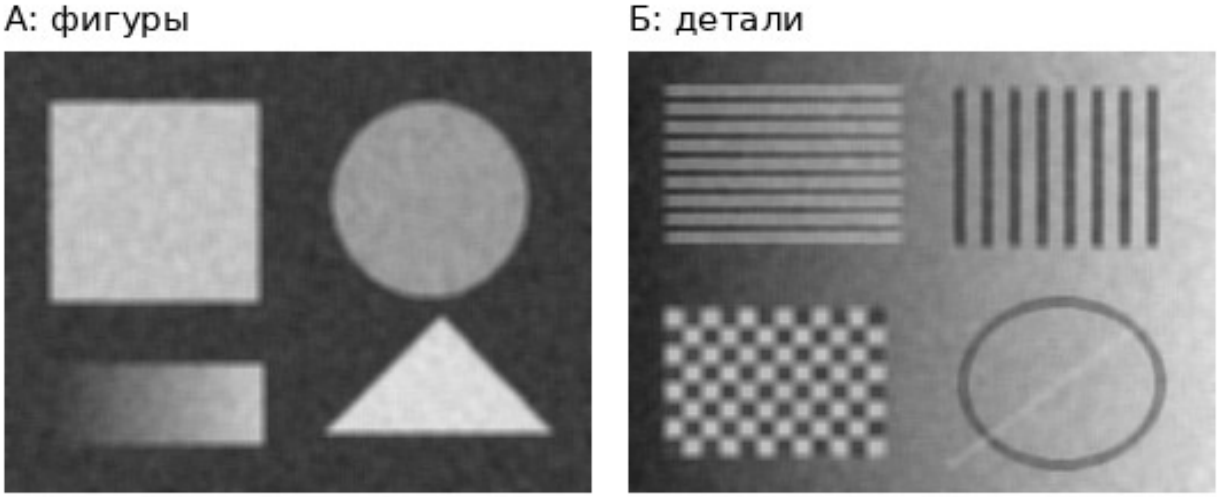



Рис. 6. Результат действия прямоугольного усредняющего фильтра 5×5 в том же расположении.

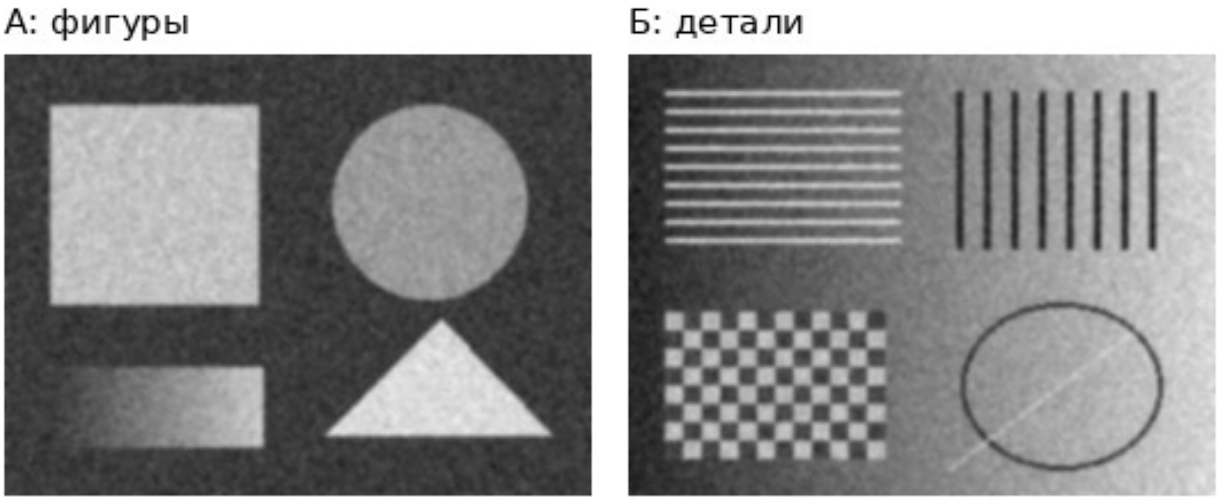



Рис. 7. Результат действия усредняющего фильтра Гаусса 5×5, σ=1.

Линейное усреднение уменьшает гауссов шум, но сопровождается размытием контрастных границ. Гауссово окно слабее учитывает удалённые пиксели, поэтому в данном эксперименте оно лучше сохраняет переходы, чем равномерное окно 5×5. Полностью избежать потери деталей при увеличении области усреднения не удаётся.

Для измерения ошибки результаты усреднения сопоставлены с чистыми эталонами. Используются показатели (17): меньшая MSE и большая PSNR означают более точное совпадение яркостей. Контурные карты так с исходным изображением не сравниваются, поскольку представляют другой вид сигнала.

$$\mathrm{MSE} = \frac{1}{WH} \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} \big(g(x,y) - f(x,y)\big)^2 \qquad (17)$$

$$\mathrm{PSNR} = 10 \log_{10} \left( \frac{255^2}{\mathrm{MSE}} \right)$$

| Метод | A: MSE | A: PSNR, дБ | Б: MSE | Б: PSNR, дБ |
|---|---|---|---|---|
| Шумный вход | 321.50 | 23.06 | 321.63 | 23.06 |
| Среднее 3×3 | 111.66 | 27.65 | 437.16 | 21.72 |
| Среднее 5×5 | 147.53 | 26.44 | 873.40 | 18.72 |
| Гаусс 5×5 | 95.38 | 28.34 | 400.27 | 22.11 |

На крупных фигурах все усредняющие методы уменьшают ошибку относительно шумного входа. Наименьшую MSE среди проверенных настроек даёт Гаусс 5×5: 95.38 вместо 321.50 у входа. Небольшие веса удалённых соседей ограничивают смешивание разных областей по сравнению с равномерным окном 5×5.

Увеличение равномерного окна с 3×3 до 5×5 делает фон более гладким, но MSE на сцене А возрастает с 111.66 до 147.53. Причиной становится размытие границ. На сцене Б размеры тонких линий сопоставимы с окном, и все три настройки повышают общую MSE относительно шумного входа. Снижение зернистости не компенсирует потерю полезных элементов.

Следовательно, результат необходимо оценивать одновременно по ровным участкам и деталям. Из трёх исследованных настроек гауссово окно 5×5 при σ=1 имеет меньшую ошибку на обеих сценах, однако это наблюдение не доказывает его превосходство при произвольных изображениях и параметрах шума.

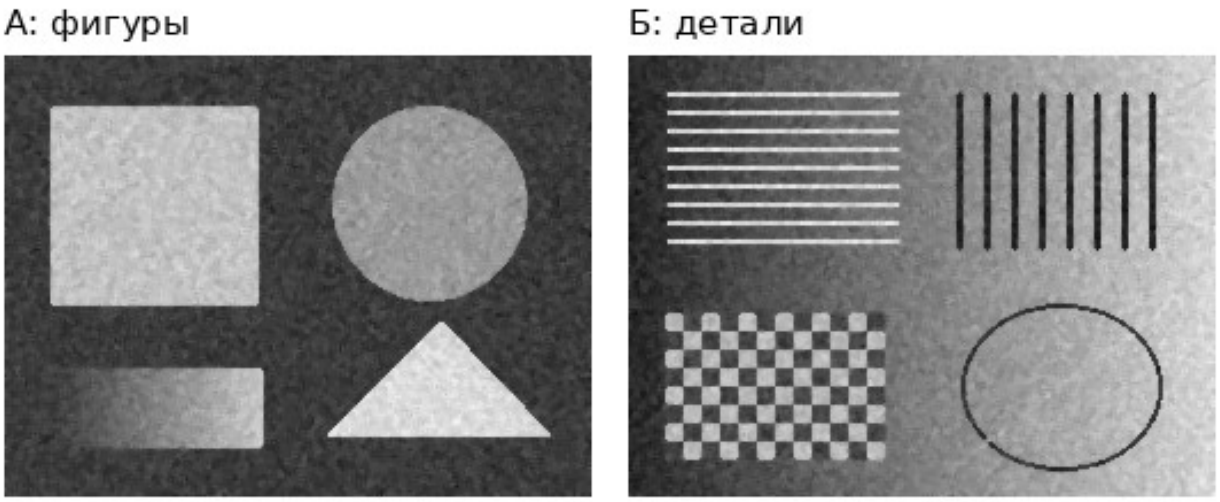



Рис. 8. Медианный фильтр 3×3: слева А, справа Б.

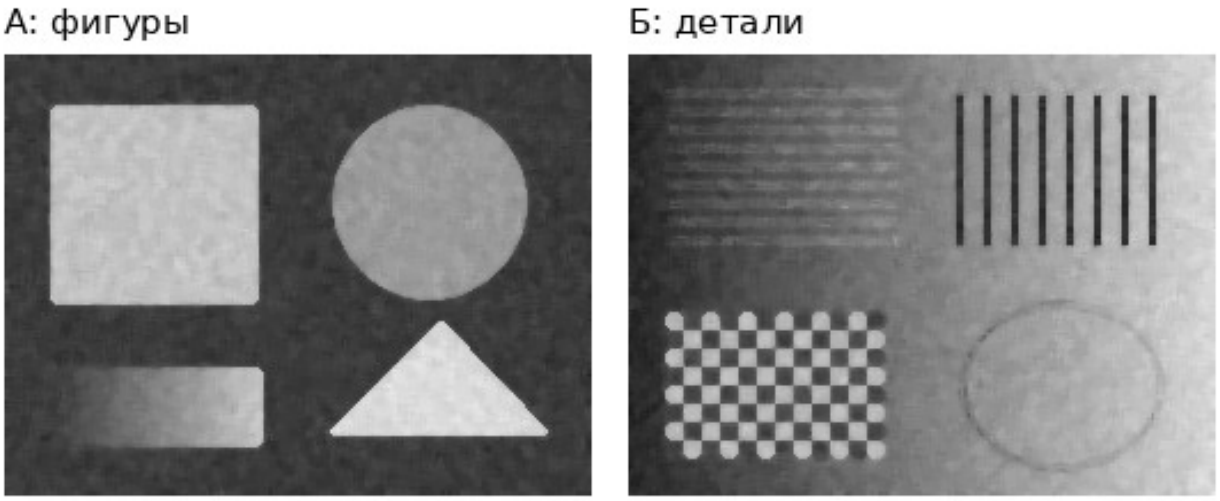



Рис. 9. Медианный фильтр 5×5: слева А, справа Б.

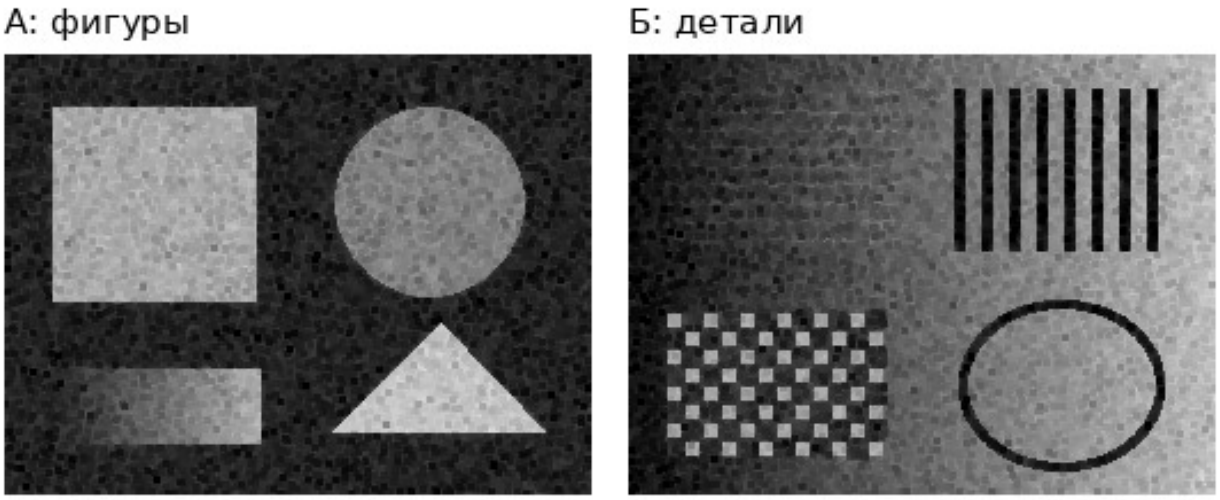



Рис. 10. Фильтр минимума 3×3: слева А, справа Б.

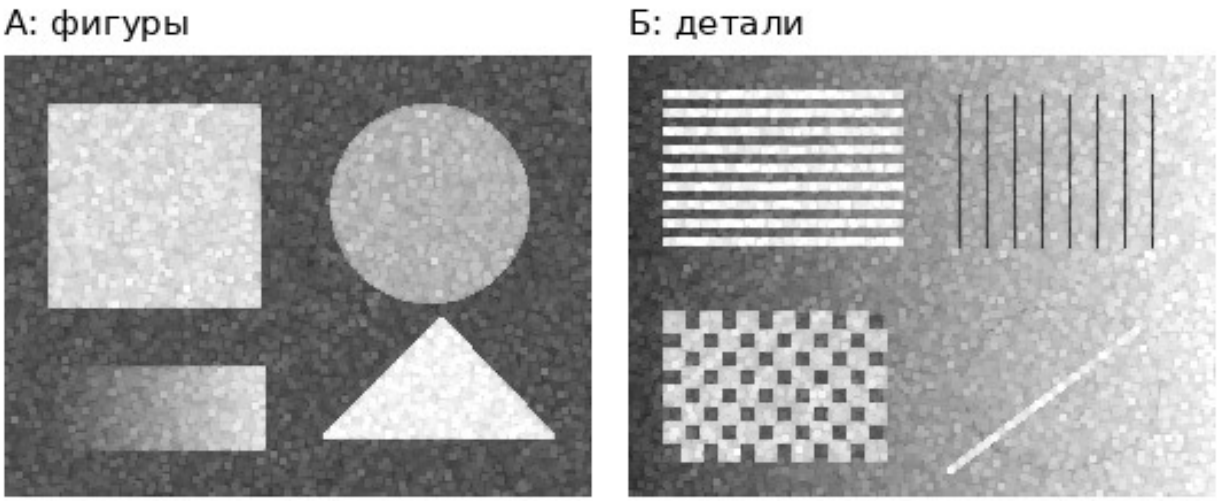



Рис. 11. Фильтр максимума 3×3: слева А, справа Б.

| Метод | А: MSE | А: PSNR, дБ | Б: MSE | Б: PSNR, дБ |
|---|---|---|---|---|
| Шумный вход | 321.50 | 23.06 | 321.63 | 23.06 |
| Медиана 3×3 | 65.27 | 29.98 | 124.90 | 27.17 |
| Медиана 5×5 | 44.62 | 31.64 | 830.54 | 18.94 |
| Минимум 3×3 | 1306.65 | 16.97 | 2975.02 | 13.40 |
| Минимум 5×5 | 2337.72 | 14.44 | 4997.93 | 11.14 |
| Максимум 3×3 | 1230.13 | 17.23 | 2866.88 | 13.56 |

| Метод | А: MSE | А: PSNR, дБ | Б: MSE | Б: PSNR, дБ |
|---|---|---|---|---|
| Максимум 5×5 | 2250.29 | 14.61 | 5346.53 | 10.85 |

На крупных фигурах медиана 5×5 даёт MSE=44.62, а на тонких деталях лучше медиана 3×3 с MSE=124.90. При увеличении окна до 5×5 MSE на сцене Б возрастает до 830.54: узкая линия занимает меньше половины окна и заменяется фоном. Выбор окна зависит от размера деталей.

Минимум затемняет изображение и сужает светлые элементы, максимум осветляет и расширяет их. Выбор экстремумов усиливает смещение яркости при гауссовом шуме. Окна 5×5 охватывают больше значений и усиливают этот эффект; численные результаты приведены в таблице. На этих сценах подходящая медиана лучше проверенных усредняющих настроек по MSE, однако универсального преимущества для любых изображений это не устанавливает.

## 5. Заключение

Реализована на языке C++17 обработка полутоновых изображений по всем трём вариантам задания.

Выполнено выделение контуров на изображении с применением следующих операторов:

1. Оператор Лапласа;
2. Оператор Робертса;
3. Оператор Собела.

Реализовано подавление гауссова шума на изображениях в пространственной области усредняющими фильтрами: арифметическим с окнами 3×3 и 5×5 и гауссовым с окном 5×5 при $\sigma=1$.

Реализована обработка изображений с гауссовым шумом порядковыми фильтрами: медианным, минимума и максимума с окнами 3×3 и 5×5.

Завершён заданный класс Image: обеспечены чтение и сохранение PNG, доступ к пикселям, поддержка одного–четырёх каналов и независимое копирование изображений. Сохранён интерфейс класса без добавления новых методов.

Подготовлены тестовые изображения, демонстрирующие работу и различия методов. Корректность программы проверена тестами, результаты сравнены с чистыми эталонами по MSE и PSNR и сохранены в отдельные файлы.

## Приложение А

Реализация операторов sobelEdges, robertsEdges и laplaceEdges. В начале листинга приведены общие вспомогательные функции и преобразование в полутоновый вид; они также используются в приложении Б.

```
#include "Filters.h"
#include <algorithm>
#include <cmath>
#include <vector>
#include <limits>
namespace {
enum class Edge { Laplace, Roberts, Sobel };
enum class Rank { Median, Minimum, Maximum };
void checkGrayscale(const Image& image) {
    if (image.width() <= 0 || image.height() <= 0 || image.nChannels()
!= 1)
           throw std::invalid_argument("Expected a nonempty grayscale
image");
}
void checkWindow(int window) {
    if (window < 1 || window > 31 || window % 2 == 0)
         throw std::invalid_argument("Window must be odd and between 1
and 31");
}
int readBorderPixel(const Image& image, int x, int y) {
    // Replicate the closest border pixel; do not add artificial black
calculateEdges.
    x = std::clamp(x, 0, image.width() - 1);
    y = std::clamp(y, 0, image.height() - 1);
    int r = 0, g = 0, b = 0;
    image.getColor(x, y, r, g, b);
    return r;
}
void writePixel(Image& image, int x, int y, double value) {
     const int v = static_cast<int>(std::lround(std::clamp(value, 0.0,
255.0)));
    image.setColor(x, y, v, v, v);
}
}
namespace Filters {
Image convertToGrayscale(const Image& input) {
    if (input.width() <= 0 || input.height() <= 0)
        throw std::invalid_argument("Empty input image");
    Image result(input.width(), input.height(), 1);
    for (int y = 0; y < input.height(); ++y)
        for (int x = 0; x < input.width(); ++x) {
            int r = 0, g = 0, b = 0;
            input.getColor(x, y, r, g, b);
```

```
                writePixel(result, x, y, input.nChannels() < 3 ? r :
0.299*r + 0.587*g + 0.114*b);
        }
    return result;
}
static Image calculateEdges(const Image& gray, Edge method) {
    checkGrayscale(gray);
    Image result(gray.width(), gray.height(), 1);
    const int sx[3][3] = {{-1,0,1}, {-2,0,2}, {-1,0,1}};
    const int sy[3][3] = {{-1,-2,-1}, {0,0,0}, {1,2,1}};
    for (int y = 0; y < gray.height(); ++y)
        for (int x = 0; x < gray.width(); ++x) {
            double value = 0;
            if (method == Edge::Laplace) {
                        value = std::abs(readBorderPixel(gray,x-1,y) +
readBorderPixel(gray,x+1,y) +
                                            readBorderPixel(gray,x,y-1) +
readBorderPixel(gray,x,y+1) - 4*readBorderPixel(gray,x,y));
            } else if (method == Edge::Roberts) {
                           const int a = readBorderPixel(gray,x,y) -
readBorderPixel(gray,x+1,y+1);
                         const int b = readBorderPixel(gray,x+1,y) -
readBorderPixel(gray,x,y+1);
                value = std::hypot(a, b);
            } else {
                int gx = 0, gy = 0;
                for (int j = -1; j <= 1; ++j)
                    for (int i = -1; i <= 1; ++i) {
                        const int p = readBorderPixel(gray,x+i,y+j);
                        gx += p * sx[j+1][i+1];
                        gy += p * sy[j+1][i+1];
                    }
                value = std::hypot(gx, gy);
            }
            writePixel(result, x, y, value);
        }
    return result;
}
Image sobelEdges(const Image& source) {
    return calculateEdges(convertToGrayscale(source), Edge::Sobel);
}
Image robertsEdges(const Image& source) {
    return calculateEdges(convertToGrayscale(source), Edge::Roberts);
}
Image laplaceEdges(const Image& source) {
    return calculateEdges(convertToGrayscale(source), Edge::Laplace);
}
}
```

## Приложение Б

Реализация усреднения meanFilter и gaussianFilter. Общий расчёт weightedAverage использует вспомогательные функции, приведённые в приложении А.

```
namespace Filters {
static Image weightedAverage(const Image& gray, int window, double
sigma) {
    checkGrayscale(gray);
    checkWindow(window);
    if (!std::isfinite(sigma) || sigma < 0)
              throw std::invalid_argument("Sigma must be finite and
nonnegative");
    const int radius = window / 2;
    std::vector<double> weights(window * window, 1.0);
    double total = 0;
    for (int j = -radius; j <= radius; ++j)
        for (int i = -radius; i <= radius; ++i) {
            double& w = weights[(j+radius)*window+i+radius];
            if (sigma > 0) {
                const double dx = i / sigma, dy = j / sigma;
                w = std::exp(-0.5 * (dx*dx + dy*dy));
            }
            total += w;
        }
    for (double& w : weights) w /= total;
    Image result(gray.width(), gray.height(), 1);
    for (int y = 0; y < gray.height(); ++y)
        for (int x = 0; x < gray.width(); ++x) {
            double sum = 0;
            for (int j = -radius; j <= radius; ++j)
                for (int i = -radius; i <= radius; ++i)
                          sum += weights[(j+radius)*window+i+radius] *
readBorderPixel(gray,x+i,y+j);
            writePixel(result,x,y,sum);
        }
    return result;
}
Image meanFilter(const Image& source, int windowSize) {
       return weightedAverage(convertToGrayscale(source), windowSize,
0.0);
}
Image gaussianFilter(const Image& source, int windowSize, double
sigma) {
    if (!std::isfinite(sigma) || sigma <= 0)
         throw std::invalid_argument("Gaussian sigma must be finite and
positive");
```

```
        return  weightedAverage(convertToGrayscale(source),  windowSize,
sigma);
}
}
```

## Приложение В

Реализация medianFilter, minimumFilter и maximumFilter. Общие функции и перечисление Rank приведены в приложении А.

```
namespace Filters {
static Image selectFromWindow(const Image& gray, int window, Rank
method) {
    checkGrayscale(gray);
    checkWindow(window);
    Image result(gray.width(), gray.height(), 1);
    const int radius = window / 2;
    std::vector<int> values(window * window);
    for (int y = 0; y < gray.height(); ++y)
        for (int x = 0; x < gray.width(); ++x) {
            int index = 0;
            for (int j = -radius; j <= radius; ++j)
                for (int i = -radius; i <= radius; ++i)
                    values[index++] = readBorderPixel(gray, x+i, y+j);
            int value = 0;
            if (method == Rank::Minimum)
                            value  =  *std::min_element(values.begin(),
values.end());
            else if (method == Rank::Maximum)
                            value  =  *std::max_element(values.begin(),
values.end());
            else {
                auto middle = values.begin() + values.size()/2;
                              std::nth_element(values.begin(),  middle,
values.end());
                value = *middle;
            }
            writePixel(result, x, y, value);
        }
    return result;
}
Image medianFilter(const Image& source, int windowSize) {
       return  selectFromWindow(convertToGrayscale(source),  windowSize,
Rank::Median);
}
Image minimumFilter(const Image& source, int windowSize) {
       return  selectFromWindow(convertToGrayscale(source),  windowSize,
Rank::Minimum);
}
Image maximumFilter(const Image& source, int windowSize) {
       return  selectFromWindow(convertToGrayscale(source),  windowSize,
Rank::Maximum);
}
}
```